



How The Khata App Was Built

A comprehensive technical breakdown of the full-stack architecture, double-entry bookkeeping engine, real-time shared kitchen synchronization, hybrid Android APK packaging, and cloud deployment pipelines.

React 19

Node.js 24 / Express 5

MongoDB & Mongoose 9

Capacitor 6 Android

Tailwind CSS 4

Vite 8

Server-Sent Events (SSE)

PWA Offline Engine

SYSTEM VERSION

v4.0.0 Production Edition

PLATFORMS

Web (PWA) + Android APK + REST API

SECURITY SPEC

Bcrypt (Salt 12) + JWT + Strict RBAC

1. Executive Summary & Product Vision

Khata is a multi-platform, full-stack financial bookkeeping and shared community ledger application. It replaces error-prone paper ledgers (traditional *Bahi-Khata*) and chaotic flatmate spreadsheets with an intuitive, mobile-first digital experience.

The system solves two distinct financial workflows in a single cohesive platform:

- **Personal & Business Ledger:** Complete double-entry customer credit (*Udhar*) and payment (*Jama*) management, daily cashbook expense registers, automated WhatsApp reminder dispatch, and instant vector PDF financial statement generation.
- **Collaborative Kitchen & Flatmate Ecosystem:** Multi-user shared flat/hostel management featuring real-time Server-Sent Events (SSE), equal bill splitting with 1-click member ledger integration, and round-robin turn-tracker duty rosters (cooking, cleaning, groceries).

Core Architectural Principle: Data Domain Isolation

Khata enforces strict boundary separation between a user's **Personal Ledger** (private shop/individual transactions) and **Shared Kitchen Groups** (collaborative flatmate expenses). The two data domains are isolated in separate database collections, state contexts, and API endpoints, preventing any inadvertent data leakage.

2. High-Level System Architecture

Khata is built on a modern client-server topology designed for high availability, offline resilience, and cross-platform native execution:



System Technology Stack Summary

Tier	Technologies	Engineering Purpose
Frontend	React 19, Vite 8, Tailwind CSS 4, Lucide React	High-performance UI rendering, zero-runtime CSS tokens, modular icon system, and instant HMR development.
Mobile Native	Capacitor 6, Android Gradle Plugin, Expo Camera	Wraps web assets into a native Android APK; integrates native hardware capabilities (Camera for QR, Haptics).
Backend API	Node.js 24, Express 5, CORS, Express-Rate-Limit	RESTful JSON services, secure token verification, route guards, and DDOS/brute-force rate limiting.
Database	MongoDB Atlas / Local MongoDB, Mongoose 9	Scalable document persistence, unique schema indexing, atomic document updates, and relational-style joins.
Security	bcryptjs, jsonwebtoken, PIN Lock Engine	Cryptographic password storage, tamper-proof session tokens, client-side financial privacy screen lock.

3. Codebase Structure & Directory Blueprint

The project is structured as a unified monorepo containing the web frontend, the backend API server, native Android container project files, and automated deployment scripts:

```
khata/
├── android/                                # Native Android Studio project generated by Capacitor 6
│   ├── app/src/main/                      # AndroidManifest.xml, MainActivity.java, App Icons
│   └── build.gradle                       # Native Android Gradle build configurations
├── server/                                # Node.js + Express 5 Production Backend
│   ├── models/                           # Mongoose Schema Definitions (User, UserData, Kitchen, etc.)
│   ├── auth.js                           # JWT issuance, token verification, and RBAC middleware
│   ├── db.js & mongodb.js                # MongoDB connection management & legacy SQLite helpers
│   ├── server.js                         # Central Express server, API routes, and SSE streaming
│   └── utils.js                          # Nanoid generator, join code creators
├── src/                                  # React 19 Frontend Client
│   ├── components/                       # Modular UI Components
│   │   ├── auth/                         # LoginView, RegisterView, LockScreenModal
│   │   ├── cashbook/                    # CashbookView (Cash in / Cash out register)
│   │   ├── customer/                   # CustomerDetail, AddCustomerModal, AddTransactionModal
│   │   ├── dashboard/                  # DashboardView (Summary cards, customer list, quick actions)
│   │   ├── kitchen/                    # Kitchen group manager, roommate cards, join modal
│   │   ├── layout/                     # MobileHeader, BottomNav (Tab navigation)
│   │   ├── qr/                         # QRPayView (Dynamic UPI QR generator)
│   │   ├── reports/                    # ReportsView (PDF statement triggers, CSV download)
│   │   ├── roster/                     # TurnTrackerView (Round-robin item duty manager)
│   │   ├── split/                      # SplitExpenseModal (Bill splitting calculation & auto-log)
│   │   └── ui/                          # Toast notifications, OnlineStatusBanner
│   ├── context/                         # React Context Providers (AuthContext, PersonalContext, KitchenContext)
│   ├── services/                        # localDb, firestoreService, syncEngine, nativeStorage
│   ├── utils/                           # pdfGenerator.js, formatters.js, translations.js, whatsapp.ts
│   ├── App.jsx                          # Application root, routing orchestration, modal controller
│   └── index.css                         # Global styles & Tailwind CSS v4 import
├── Build-Android-APK.bat                 # 1-Click batch build: Vite build -> Cap sync -> Gradle APK
├── Start-Khata.bat                       # 1-Click local launch: Concurrently runs server + Vite
├── capacitor.config.json                 # Capacitor Android native wrapper configuration
└── package.json                          # Root dependencies & build scripts
```

4. Database Schema & Data Modeling (MongoDB)

Khata uses **MongoDB via Mongoose 9**. The database is organized into distinct collections to optimize querying speed and maintain clean separation of concerns:

Collection	Primary Keys & Indexes	Fields & Payload Description
users	<code>_id (usr_*)</code> , <code>phone (unique)</code> , <code>email (unique)</code>	Core authentication entity. Stores name, phone, email, hashed password (<code>password_hash</code>), and primary shop name.
user_data	<code>_id (matches user._id)</code>	Embedded document containing the user's business profile, multi-books, customer list, transaction ledgers, cashbook records, and preferences.

Collection	Primary Keys & Indexes	Fields & Payload Description
kitchen_groups	<code>_id (kg_*)</code> , <code>join_code</code> (unique)	Shared kitchen/flatmate space. Stores group name, 6-character alphanumeric join code (e.g. <code>KHT-892</code>), owner ID, and max member limit.
kitchen_members	<code>_id (km_*)</code> , Compound: <code>(group_id, user_id)</code>	Membership join record. Stores user ID reference (null for manual offline members), display name, role (<code>OWNER ADMIN MEMBER</code>), and status (<code>active paused</code>).
kitchen_data	<code>_id (matches group._id)</code>	Shared community data: contains the roster duties array and the shared kitchen cashbook expenses array.
password_resets	<code>_id (pr_*)</code> , TTL index: <code>expires_at</code>	Stores 6-digit one-time password (OTP) codes with a 15-minute expiration timestamp and usage flag.

5. Backend Architecture & API Security

The backend is powered by **Express 5** running on **Node.js 24**. It adheres to REST architectural standards, stateless JWT session verification, and proactive defense-in-depth principles.

5.1 Authentication & Password Security

- **Bcrypt Password Hashing:** Passwords are never stored in plain text. Every password is salted and hashed using `bcryptjs` with an industry-standard work factor of 12 rounds.
- **Legacy Password Migration:** The authentication middleware automatically detects older legacy password strings, verifies them securely, and silently upgrades them to bcrypt hashes upon successful authentication without interrupting the user.
- **Stateless JWT Tokens:** Upon authentication, an HMAC-SHA256 signed JSON Web Token is issued containing the user ID. Protected routes inspect the `Authorization: Bearer <token>` header.

5.2 Rate Limiting & Protection Against Abuse

To prevent brute-force attacks and credential stuffing, the server deploys two independent rate-limiting tiers via `express-rate-limit`:

- `authLimiter`: Restricts registration, login, and password reset endpoints to a maximum of **15 requests per 15-minute window** per IP address.
- `generalLimiter`: Limits general API operations to **120 requests per minute**, safeguarding against automated scraping or denial-of-service attempts.

5.3 Real-Time Server-Sent Events (SSE) Hub

Instead of introducing heavy WebSocket connection overhead, Khata utilizes lightweight **Server-Sent Events (SSE)** at `GET /api/kitchen/:groupId/events` to stream real-time updates to connected roommates:

```
// Server-Sent Events implementation in server/server.js
const sseClients = new Map(); // groupId -> Set<Response>

app.get("/api/kitchen/:groupId/events", authenticateToken, requireKitchenMember(), (req, res) => {
  res.setHeader("Content-Type", "text/event-stream");
  res.setHeader("Cache-Control", "no-cache");
  res.setHeader("Connection", "keep-alive");
  res.flushHeaders();

  if (!sseClients.has(groupId)) sseClients.set(groupId, new Set());
  sseClients.get(groupId).add(res);

  // Periodic heartbeat every 30 seconds to maintain connection through NAT firewalls
  const heartbeat = setInterval(() => {
    try { res.write(": ping\n\n"); } catch { clearInterval(heartbeat); }
  }, 30000);

  req.on("close", () => {
    clearInterval(heartbeat);
    sseClients.get(groupId)?.delete(res);
  });
});
```

```
});
```

5.4 Complete REST API Endpoints Reference

Method & Route	Auth Required	Description & Payload
POST /api/auth/register	Public	Registers user with phone/email, name, shop name, and bcrypt-hashed password.
POST /api/auth/login	Public	Authenticates credentials, returns signed JWT token and user profile object.
POST /api/auth/forgot-password	Public	Generates a 6-digit OTP code with 15-minute expiration for password reset.
POST /api/auth/reset-password	Public	Validates OTP code and commits new password hash to user profile.
GET /api/personal	Bearer Token	Fetches the authenticated user's complete ledger, customers, and cashbook data.
POST /api/personal/sync	Bearer Token	Atomically synchronizes updated customers, transactions, or cashbook records.
POST /api/kitchen/create	Bearer Token	Creates a new shared kitchen group, assigns owner role, and generates join code.
POST /api/kitchen/join	Bearer Token	Joins existing kitchen group via 6-character uppercase join code.
POST /api/kitchen/:groupId/sync	Member Token	Updates group roster or cashbook; broadcasts live SSE event to all connected members.
POST /api/kitchen/:groupId/member/toggle-pause	Member Token	Toggles member status between active and paused (for vacation/absence).

6. Frontend Architecture & State Management

Khata's frontend is constructed using **React 19**, **Vite 8**, and **Tailwind CSS 4**. The design philosophy centers around high responsiveness, tactile mobile feedback, and clear domain separation.

6.1 Triple Context Architecture

To prevent monolithic state bloat and eliminate unwanted cross-component re-renders, application state is divided into three distinct React Context layers:

1. AuthContext **SECURITY**

Manages user authentication lifecycle, JWT token persistence in LocalStorage, login/register mutations, and the client-side PIN security lockscreen (`isLocked`).

2. PersonalContext **LEDGER**

Encapsulates private shop/individual ledger data: customer records, udhar/jama transactions, daily cashbook, and business profile. Employs user-scoped localStorage keys (`khata_{userId}_{key}`) for offline reliability.

3. KitchenContext **REALTIME**

Manages collaborative kitchen groups, roommate memberships, duty rosters, and shared cashbook expenses. Subscribes to the live SSE stream with an automatic heartbeat and fallback polling mechanism.

4. Offline-First Resilience **RELIABILITY**

If network connectivity drops or the cloud server is unreachable, the app automatically transitions to local cache mode, rendering an amber status banner while queuing changes for synchronization.

6.2 Core UI View Hierarchy & Routing

Rather than relying on bulky URL routers that add bundle size in mobile environments, Khata utilizes a responsive state-driven view controller within `src/App.jsx`:

- **Dashboard** (`DashboardView.jsx`): Financial summary cards (Total to Collect, Total to Pay, Daily Cash balance), customer search and filtering bar, quick transaction launchers, and customer list.
- **Customer Ledger Detail** (`CustomerDetail.jsx`): Drill-down view into an individual customer's transaction history, WhatsApp reminder triggers, and PDF account statement download.
- **Cashbook Register** (`CashbookView.jsx`): Running daily ledger of cash income and cash expenses with running cash-in-hand calculation.
- **Turn Tracker** (`TurnTrackerView.jsx`): Duty manager displaying whose turn it is next for cooking, cleaning, or grocery shopping with WhatsApp rotation alerts.
- **Dynamic QR & Pay** (`QRPayView.jsx`): Real-time SVG payment QR code generation with dynamic amount and merchant UPI parameters.
- **Financial Reports** (`ReportsView.jsx`): Statements hub allowing one-click export of complete PDF statements or Excel-compatible CSV balance sheets.
- **Settings & Security** (`SettingsView.jsx`): Business branding configuration, PIN lock setup, language toggle (English/Hindi), and JSON data backup/restore.

6.3 Design Tokens & Styling (Tailwind CSS 4)

The user interface uses a tailored color palette specifically optimized for financial clarity:

- **Emerald Green** (`#10b981` / `#059669`): Denotes Jama (payments received), positive cash inflow, and credit assets.
- **Rose Red** (`#ef4444` / `#dc2626`): Denotes Udhar (credit extended), pending customer liabilities, and cash outflows.
- **Slate Dark** (`#0f172a` / `#1e293b`): Used for high-contrast header cards, elevated modal sheets, and tab navigation.
- **Tactile Micro-interactions**: Active scaling (`active:scale-95`), smooth CSS transitions, and celebratory confetti animations (`canvas-confetti`) when full payments are settled.

7. Core Business Logic & Algorithms

7.1 Double-Entry Balance Calculation Engine

At the heart of Khata is the balance calculation algorithm implemented in `src/utils/formatters.js`. For any customer account, transactions are categorized into two types:

- **Gave (Udhar):** Credit extended to the customer (they owe you money).
- **Got (Jama):** Payment received from the customer (they paid you money).

```
export const calculateCustomerBalance = (transactions = []) => {
  let totalGave = 0; // Udhar given by shopkeeper
  let totalGot = 0; // Jama paid by customer

  for (const tx of transactions) {
    const amt = Number(tx.amount || 0);
    if (tx.type === "gave") totalGave += amt;
    else if (tx.type === "got") totalGot += amt;
  }

  const diff = totalGave - totalGot;

  if (diff > 0) {
    return { balance: diff, status: "get" }; // Shopkeeper will collect money (Asset)
  } else if (diff < 0) {
    return { balance: Math.abs(diff), status: "give" }; // Customer has advance credit (Liability)
  }
  return { balance: 0, status: "settled" }; // Account is fully settled
};
```

7.2 Turn Tracker Round-Robin Rotation Algorithm

In shared flats or hostel groups, roommates share duties such as cooking, grocery shopping, or water can refills. The turn progression algorithm guarantees fair rotation while skipping members who are temporarily marked as `paused` (e.g. traveling or on leave):

```
function advanceDutyTurn(dutyItem, members) {
  const activeMembers = dutyItem.memberIds
    .map(id => members.find(m => m.id === id || m.userId === id))
    .filter(m => m && m.status !== "paused"); // Skip absent/paused roommates

  if (activeMembers.length === 0) return dutyItem;

  const nextIndex = (dutyItem.currentTurnIndex + 1) % activeMembers.length;
  const currentPerson = activeMembers[nextIndex % activeMembers.length];

  return {
    ...dutyItem,
    currentTurnIndex: nextIndex,
    lastBroughtBy: currentPerson.name,
    lastBroughtDate: new Date().toISOString(),
    history: [
      { person: currentPerson.name, date: new Date().toISOString() },
    ],
  };
}
```

```
...(dutyItem.history || [])
]
};
}
```

7.3 Shared Bill Splitting & Ledger Auto-Injection

When an expense occurs in a kitchen group (e.g. a ₹1,200 grocery bill), the split calculator automatically computes the per-person liability (`Math.round(total / count)`) and gives the user the option to automatically log an Udhar transaction into each member's personal ledger with a single click.

7.4 Dynamic UPI Intent & QR Code Protocol

Khata integrates the standard National Payments Corporation of India (NPCI) UPI Intent specification. In `QRPayView.jsx`, payment links are assembled according to the standard URI scheme:

```
upi://pay?pa={merchantUpiId}&pn={encode(businessName)}&am={amount}&cu=INR&tn={encode(note)}
```

This URI is fed directly into `qrcode.react` to generate a crisp vector QR code compatible with Google Pay, PhonePe, Paytm, BHIM, and all mobile banking applications.

7.5 Client-Side Vector PDF Engine (`jspdf` + `jspdf-autotable`)

Unlike apps that rely on slow server-side rendering pipelines (like Puppeteer), Khata renders complete financial statements directly in the browser using client-side JavaScript. This enables instant statement generation even when completely offline:

- Creates branded vector headers with merchant business name, address, GSTIN, and phone number.
- Calculates summary boxes highlighting net receivables vs. payables.
- Constructs styled data tables with alternating row shading, date formatters, and totals footers.
- Exports clean, print-ready PDF files directly into the user's downloads folder.

8. Mobile Native Shell & Packaging (Android APK)

Khata is engineered to run seamlessly as both a responsive web application and a native Android APK. This is achieved using **Capacitor 6**.

8.1 Capacitor Bridge Architecture

Capacitor bridges the React web bundle (`dist/`) inside a native Android `WebView` wrapper, providing native runtime performance without requiring a separate React Native rewrite:

- `capacitor.config.json` : Declares the native package ID (`com.khata.app`), application title, and assets source directory.
- `android/` Directory: Fully configured Android Studio project featuring `MainActivity.java` and Gradle build scripts.
- **Hardware Permissions:** Android manifest enables camera access (`android.permission.CAMERA`) for scanning UPI and invite QR codes, alongside storage and network state permissions.

8.2 Automated 3-Stage APK Build Pipeline

Building the Android APK is entirely automated via `Build-Android-APK.bat` :

```
# Stage 1: Build production web bundle via Vite
call npm run build

# Stage 2: Sync web assets into native Android directory
call npx cap sync android

# Stage 3: Compile debug APK via Gradle Wrapper
cd android
call gradlew.bat assembleDebug

# Output APK: android/app/build/outputs/apk/debug/app-debug.apk
```

9. DevOps, Deployment & Production Readiness

The application is configured with multi-cloud deployment scripts for hosting the frontend and containerizing the backend:

Target Environment	Configuration File	Deployment Workflow
Docker / Compose	<code>Dockerfile</code> , <code>compose.yaml</code>	Containerizes the Node.js API with health checks and environment variable injection.
Render Cloud	<code>render.yaml</code>	Deploys the production API web service with continuous deployment from GitHub.
Firebase Hosting	<code>firebase.json</code> , <code>Deploy-Firebase.bat</code>	Builds the optimized web bundle and pushes to Firebase's global CDN with SPA rewrite rules.

Target Environment	Configuration File	Deployment Workflow
Local Development	<code>Start-Khata.bat</code>	Concurrently spawns the backend server (Port 5000) and the Vite development server (Port 5173).

10. Step-by-Step Blueprint: How to Rebuild This App

- 1. Initialize Project:** Scaffold a React + Vite application with TypeScript and Tailwind CSS v4.
- 2. Configure Backend Server:** Set up Express 5 with CORS, Rate Limiting, and MongoDB Mongoose connection.
- 3. Implement Authentication:** Build User schema with Bcrypt password hashing and JWT token issuance.
- 4. Construct Personal Ledger Context:** Implement double-entry calculations (`gave` / `got`) with user-scoped LocalStorage synchronization.
- 5. Build Core UI Screens:** Create Dashboard, Customer Details, Add Entry modals, and Cashbook register.
- 6. Implement Kitchen Group & SSE:** Build shared space models and Server-Sent Events broadcasting for flatmates.
- 7. Integrate PDF & QR Engines:** Implement `jspdf` report generation and dynamic UPI payment QR codes.
- 8. Wrap with Capacitor:** Initialize Capacitor Android wrapper and compile the native APK using Gradle.

Summary & Engineering Takeaway

The Khata application demonstrates how modern web technologies (React 19, Vite, Tailwind CSS 4) can be paired with an efficient Node.js/Express/MongoDB backend and wrapped in Capacitor to deliver a robust, cross-platform financial application that runs seamlessly on web, desktop, and mobile devices.